LoRA Fine-Tuning Assignment

Objective

Apply what you learned in the SFT + LoRA tutorial to fine-tune **TinyLlama-1.1B** on the **databricks-dolly-15k** dataset using LoRA.

What is provided

- All installation, import, and utility cells are provided.
- Each task cell has a comment block describing exactly what you need to write.

What you need to do

Fill in every cell marked with `# YOUR CODE HERE`. There are **8 tasks** in total.

Prerequisites

- You have completed the SFT + LoRA tutorial notebook.
- Runtime: **GPU** (T4 is sufficient).

Part 0 — Setup (provided)

In [16]:
```
!pip install -U transformers trl peft datasets accelerate -q
```

In [17]:
```
import torch
import gc
import os
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, PeftModel, get_peft_model
from trl import SFTConfig, SFTTrainer

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: Tesla T4
```

Utility functions (provided)

In [18]:
```
def clear_memory():
    """Free GPU memory between experiments."""
    gc.collect()
    torch.cuda.empty_cache()
    if torch.cuda.is_available():
        print(f"GPU allocated: {torch.cuda.memory_allocated() / 1024**3:.2f} GB")
        print(f"GPU reserved:  {torch.cuda.memory_reserved() / 1024**3:.2f} GB")


def print_trainable_params(model):
    """Display trainable vs total parameter counts."""
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    print(f"Trainable parameters: {trainable:>12,}  ({100 * trainable / total:.4f}%)")
    print(f"Total parameters:     {total:>12,}")


def generate_response(model, tokenizer, prompt, max_new_tokens=256):
    """Generate a response from a model given a prompt string."""
    messages = [{"role": "user", "content": prompt}]
    input_text = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            temperature=0.7,
            top_p=0.9,
            do_sample=True,
            pad_token_id=tokenizer.pad_token_id,
        )
```

```
    response = tokenizer.decode(outputs[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True)
    return response
```

Part 1 — Configuration

Task 1: Define all hyperparameters

Set the following configuration variables:

Variable	Value
`MODEL_NAME`	`"TinyLlama/TinyLlama-1.1B-Chat-v1.0"`
`DATASET_NAME`	`"databricks/databricks-dolly-15k"`
`NUM_TRAIN_SAMPLES`	`1000`
`MAX_SEQ_LENGTH`	`512`
`LORA_R`	`8`
`LORA_ALPHA`	`16`
`LORA_DROPOUT`	`0.05`
`LORA_TARGET_MODULES`	`["q_proj", "v_proj"]` (attention only)
`LEARNING_RATE`	`2e-4`
`NUM_EPOCHS`	`1`
`BATCH_SIZE`	`4`
`GRAD_ACCUM_STEPS`	`4`
`OUTPUT_DIR`	`"./sft-lora-tinyllama"`

In [19]:
```
MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
DATASET_NAME = "databricks/databricks-dolly-15k"
NUM_TRAIN_SAMPLES = 1000
MAX_SEQ_LENGTH = 512
LORA_R = 8
LORA_ALPHA = 16
LORA_DROPOUT = 0.05
LORA_TARGET_MODULES = ["q_proj", "v_proj"]
LEARNING_RATE = 2e-4
NUM_EPOCHS = 1
BATCH_SIZE = 4
GRAD_ACCUM_STEPS = 4
OUTPUT_DIR = "./sft-lora-tinyllama"
```

Part 2 — Load Model and Tokenizer

Task 2: Load the tokenizer and base model

1. Load the tokenizer from `MODEL_NAME`.
2. If the tokenizer has no pad token, set it to the eos token.
3. Load the model in `float16` with `device_map="auto"`.
4. Print the total parameter count.

In [20]:
```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token


model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Print total parameter count
total_params = sum(p.numel() for p in model.parameters())

print(f"Model: {MODEL_NAME}")
print(f"Total parameters: {total_params:,}")
print(f"Model dtype: {model.dtype}")
print(f"Model device: {model.device}")
print(f"Pad token: {tokenizer.pad_token}")
print(f"EOS token: {tokenizer.eos_token}")
```

Loading weights: 100% 201/201 [00:11<00:00, 17.07it/s]

```
Model: TinyLlama/TinyLlama-1.1B-Chat-v1.0
Total parameters: 1,100,048,384
Model dtype: torch.float16
Model device: cuda:0
Pad token: </s>
EOS token: </s>
```

Part 3 — Test Base Model

Task 3: Generate responses from the base model

Use the provided `generate_response` function with these test prompts:

1. `"Explain what a neural network is in 2-3 sentences."`
2. `"What are three tips for better sleep?"`

Store the responses in a list called `base_responses` for later comparison.

In [21]:
```python
test_prompts = [
    "Explain what machine learning is in simple terms.",
    "Write a short professional email requesting a meeting.",
    "What are three benefits of regular exercise?",
    "Explain the difference between a list and a tuple in Python."
]

base_responses = []

model.eval()

for prompt in test_prompts:
    messages = [
        {"role": "user", "content": prompt}
    ]


    formatted_prompt = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer(
        formatted_prompt,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=150,
            do_sample=True,
            temperature=0.7,
            top_p=0.9,
            pad_token_id=tokenizer.eos_token_id
        )


    generated_tokens = outputs[0][inputs["input_ids"].shape[1]:]

    response = tokenizer.decode(
        generated_tokens,
        skip_special_tokens=True
    ).strip()

    base_responses.append(response)

    print("=" * 80)
    print(f"PROMPT:\n{prompt}")
    print(f"\nBASE MODEL RESPONSE:\n{response}")
    print()
```

```
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
```

```
================================================================================
PROMPT:
Explain what machine learning is in simple terms.

BASE MODEL RESPONSE:
Machine learning (ML) is a field of artificial intelligence that involves the use of algorithms and data to learn and
improve without being explicitly programmed. It is an iterative process that allows machines to learn and improve on
their own without being explicitly programmed.

In simple terms, machine learning involves developing algorithms that can learn from data, making predictions based o
n the data, and improve over time. The process is iterative and involves several stages, including data collection, f
eature engineering, model selection, training, and testing.

Some examples of machine learning applications are:

1. Predictive maintenance: Using machine learning algorithms, companies can predict when their machinery will break d
own, allowing them to proactively schedule maintenance.

[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take
precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/e
n/main_classes/text_generation)
================================================================================
PROMPT:
Write a short professional email requesting a meeting.

BASE MODEL RESPONSE:
Subject: Request for a Meeting

Dear [Recipient Name],

I hope this email finds you well. I am writing to request a meeting with you at our company to discuss the following:

[Proposed topic]

This meeting will be a great opportunity for us to discuss the topic in detail, share our respective perspectives, an
d come up with a solution that meets both parties' needs.

I am confident that our meeting will be productive and fruitful, and I am looking forward to discussing this matter w
ith you.

Please let me know if you are available to meet with me on [date] at [time]. If you cannot make it, please let me kno
w

[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take
precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/e
n/main_classes/text_generation)
================================================================================
PROMPT:
What are three benefits of regular exercise?

BASE MODEL RESPONSE:
1. Increases metabolism: Regular exercise increases your metabolism, which means your body burns calories at a faster
rate. This can help you lose weight and maintain a healthy weight over time.

2. Improves cardiovascular health: Regular exercise can help improve your cardiovascular health by increasing blood f
low to your muscles and reducing the risk of heart disease.

3. Boosts mood: Regular exercise has been shown to have a positive impact on mood and overall well-being. It can help
reduce stress and improve overall happiness.

These benefits are all part of the "benefits of exercise" that you can

================================================================================
PROMPT:
Explain the difference between a list and a tuple in Python.

BASE MODEL RESPONSE:
In Python, a list is a collection of items of the same type, where each item is stored in a separate position in the
list. In contrast, a tuple is a collection of items of the same type, where each item is stored in a single position
in the tuple.

Here are some examples of how they differ:

1. List vs. Tuple:
- Lists are immutable (meaning they cannot be modified after being created)
- Tuples are mutable (meaning they can be modified after being created)

2. Length and size of a list:
- A list can have a fixed length, but it cannot grow beyond that length
- A tuple can grow as long as
```

Part 4 — Prepare the Dataset

Task 4: Load and format the Dolly dataset

The Dolly dataset has columns: `instruction`, `context`, `response`, `category`.

1. Load the dataset (`split="train"`).
2. Write a function `format_dolly_to_chat(example)` that converts each example to:

   ```
   {"messages": [
       {"role": "user", "content": <instruction + context if present>},
       {"role": "assistant", "content": <response>}
   ]}
   ```

   If `context` is non-empty, concatenate it to the instruction with a newline separator.
3. Shuffle with `seed=42`, select `NUM_TRAIN_SAMPLES` examples, and apply the formatting function.
4. Print the size of the formatted dataset and one example.

In [22]:
```python
# Load the Dolly dataset
dataset = load_dataset(DATASET_NAME, split="train")

# Convert each Dolly example into chat format
def format_dolly_to_chat(example):
    instruction = example["instruction"]
    context = example["context"]
    response = example["response"]

    # Add context to the instruction only when context is present
    if context and context.strip():
        user_content = instruction + "\n" + context
    else:
        user_content = instruction

    return {
        "messages": [
            {
                "role": "user",
                "content": user_content
            },
            {
                "role": "assistant",
                "content": response
            }
        ]
    }

# Shuffle, select training subset, and format
train_dataset = (
    dataset
    .shuffle(seed=42)
    .select(range(NUM_TRAIN_SAMPLES))
    .map(format_dolly_to_chat)
)

# Verify
print(f"Formatted dataset size: {len(train_dataset)}")
print("\nExample:")
print(train_dataset[0]["messages"])
```

```
Formatted dataset size: 1000
Example:
[{'role': 'user', 'content': 'Who were the children of the legendary Garth Greenhand, the High King of the First Men
in the series A Song of Ice and Fire?'}, {'role': 'assistant', 'content': 'Garth the Gardener, John the Oak, Gilbert
of the Vines, Brandon of the Bloody Blade, Foss the Archer, Owen Oakenshield, Harlon the Hunter, Herndon of the Horn,
Bors the Breaker, Florys the Fox, Maris the Maid, Rose of the Red Lake, Ellyn Ever Sweet, Rowan Gold-Tree'}]
```

Part 5 — Configure LoRA

Task 5: Create the LoRA config and inspect trainable parameters

1. Create a `LoraConfig` using the hyperparameters from Task 1. Set `bias="none"` and `task_type="CAUSAL_LM"`.
2. Apply it to the model with `get_peft_model`.
3. Use `print_trainable_params` to display the trainable vs total parameters.
4. Print the LoRA-wrapped structure of one attention layer (e.g., `q_proj` of layer 0).

In [23]:
```python
# Create LoRA configuration
peft_config = LoraConfig(
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    lora_dropout=LORA_DROPOUT,
    target_modules=LORA_TARGET_MODULES,
    bias="none",
    task_type="CAUSAL_LM"
)
```

```
# Apply LoRA to the base model
model = get_peft_model(model, peft_config)

# Display trainable vs total parameters
print_trainable_params(model)

# Inspect q_proj of the first attention layer
print("\nLoRA-wrapped q_proj of layer 0:")
print(model.base_model.model.model.layers[0].self_attn.q_proj)
```

```
Trainable parameters:    1,126,400  (0.1023%)
Total parameters:     1,101,174,784

LoRA-wrapped q_proj of layer 0:
lora.Linear(
  (base_layer): Linear(in_features=2048, out_features=2048, bias=False)
  (lora_dropout): ModuleDict(
    (default): Dropout(p=0.05, inplace=False)
  )
  (lora_A): ModuleDict(
    (default): Linear(in_features=2048, out_features=8, bias=False)
  )
  (lora_B): ModuleDict(
    (default): Linear(in_features=8, out_features=2048, bias=False)
  )
  (lora_embedding_A): ParameterDict()
  (lora_embedding_B): ParameterDict()
  (lora_magnitude_vector): ModuleDict()
)
```

Part 6 — Train

Task 6: Set up SFTTrainer and run training

1. Delete the PEFT-wrapped model and call `clear_memory()`. Reload the base model fresh (SFTTrainer applies LoRA itself via `peft_config`).
2. Create an `SFTConfig` with the hyperparameters from Task 1. Include:
   - Cosine LR scheduler, warmup ratio 0.03, weight decay 0.01
   - `gradient_checkpointing=True`
   - bf16 if supported, else fp16
   - `max_length=MAX_SEQ_LENGTH`
   - Logging every 10 steps
3. Create an `SFTTrainer` with the model, training args, dataset, tokenizer, and `peft_config`.
4. Call `trainer.train()` and print the final loss.

In [24]:
```
!pip install -U "torchao>=0.16.0"
```

```
Requirement already satisfied: torchao>=0.16.0 in /usr/local/lib/python3.12/dist-packages (0.18.0)
```

In [ ]:
```
# Clean up the PEFT-wrapped model from Task 5
del model
clear_memory()

# Reload a fresh base model
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    dtype=torch.float16,
    device_map="auto"
)

# Check whether BF16 is supported
use_bf16 = torch.cuda.is_available() and torch.cuda.is_bf16_supported()

print(f"Using BF16: {use_bf16}")
print(f"Using FP16: {not use_bf16}")

# Training configuration
training_args = SFTConfig(
    output_dir=OUTPUT_DIR,

    num_train_epochs=NUM_EPOCHS,

    per_device_train_batch_size=BATCH_SIZE,
    gradient_accumulation_steps=GRAD_ACCUM_STEPS,

    learning_rate=LEARNING_RATE,
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    weight_decay=0.01,

    gradient_checkpointing=True,

    bf16=use_bf16,
```

```
    fp16=not use_bf16,

    max_length=MAX_SEQ_LENGTH,

    logging_steps=10,
    save_strategy="no",

    report_to="none"
)

# LoRA configuration
peft_config = LoraConfig(
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    lora_dropout=LORA_DROPOUT,
    target_modules=LORA_TARGET_MODULES,
    bias="none",
    task_type="CAUSAL_LM"
)

# Create trainer
trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    processing_class=tokenizer,
    peft_config=peft_config
)

# Start training
train_result = trainer.train()

print(f"\nFinal training loss: {train_result.training_loss}")
```

```
GPU allocated: 4.13 GB
GPU reserved:  4.18 GB
```

Loading weights: 100% 201/201 [00:10<00:00, 21.35it/s]

```
[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Using BF16: True
Using FP16: False
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'pad_token_id': 2}.
```

[63/63 17:05, Epoch 1/1]

Step	Training Loss
10	2.112326
20	1.881704
30	1.762429
40	1.736321
50	1.782178
60	1.725159

```
Final training loss: 1.832731352912055
```

Part 7 — Save the Adapter

Task 7: Save the LoRA adapter and print its size

1. Save the trained model to `f"{OUTPUT_DIR}/final-adapter"`.
2. Also save the tokenizer to the same path.
3. Compute and print the adapter size in MB.

In [26]:
```
# Path for the final LoRA adapter
adapter_path = f"{OUTPUT_DIR}/final-adapter"

# Save trained LoRA adapter
trainer.save_model(adapter_path)

# Save tokenizer
tokenizer.save_pretrained(adapter_path)

# Calculate total adapter directory size
total_size_bytes = 0

for root, dirs, files in os.walk(adapter_path):
    for file in files:
        file_path = os.path.join(root, file)
        total_size_bytes += os.path.getsize(file_path)
```

```
total_size_mb = total_size_bytes / (1024 ** 2)

print(f"Adapter saved to: {adapter_path}")
print(f"Adapter size: {total_size_mb:.2f} MB")
```

```
Adapter saved to: ./sft-lora-tinyllama/final-adapter
Adapter size: 7.77 MB
```

Part 8 — Evaluate

Task 8: Compare base vs fine-tuned responses

1. Use `trainer.model` (already has LoRA weights) to generate responses for the same `test_prompts`.
2. Print a side-by-side comparison of base vs fine-tuned responses.

In [27]:
```
# Use the fine-tuned LoRA model
finetuned_model = trainer.model
finetuned_model.eval()

finetuned_responses = []

# Generate responses using the same test prompts
for prompt in test_prompts:
    messages = [
        {"role": "user", "content": prompt}
    ]

    # Apply TinyLlama chat template
    formatted_prompt = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer(
        formatted_prompt,
        return_tensors="pt"
    ).to(finetuned_model.device)

    with torch.no_grad():
        outputs = finetuned_model.generate(
            **inputs,
            max_new_tokens=150,
            do_sample=True,
            temperature=0.7,
            top_p=0.9,
            pad_token_id=tokenizer.eos_token_id
        )

    # Decode only newly generated tokens
    generated_tokens = outputs[0][inputs["input_ids"].shape[1]:]

    response = tokenizer.decode(
        generated_tokens,
        skip_special_tokens=True
    ).strip()

    finetuned_responses.append(response)


# Print side-by-side comparison
for i, prompt in enumerate(test_prompts):
    print("=" * 100)
    print(f"PROMPT:\n{prompt}")

    print("\n--- BASE MODEL ---")
    print(base_responses[i])

    print("\n--- FINE-TUNED MODEL ---")
    print(finetuned_responses[i])

    print()
```

```
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=150) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
```

```
================================================================================================
PROMPT:
Explain what machine learning is in simple terms.

--- BASE MODEL ---
Machine learning (ML) is a field of artificial intelligence that involves the use of algorithms and data to learn and
improve without being explicitly programmed. It is an iterative process that allows machines to learn and improve on
their own without being explicitly programmed.

In simple terms, machine learning involves developing algorithms that can learn from data, making predictions based o
n the data, and improve over time. The process is iterative and involves several stages, including data collection, f
eature engineering, model selection, training, and testing.

Some examples of machine learning applications are:

1. Predictive maintenance: Using machine learning algorithms, companies can predict when their machinery will break d
own, allowing them to proactively schedule maintenance.

--- FINE-TUNED MODEL ---
Machine Learning is a field of computer science that is concerned with developing and using algorithms and models to
automate and optimize various forms of decision-making. Machine Learning is used in various areas of science and engi
neering, such as image recognition, natural language processing, and healthcare. The goal of machine learning is to b
uild systems that can learn from data and improve their performance over time.

================================================================================================
PROMPT:
Write a short professional email requesting a meeting.

--- BASE MODEL ---
Subject: Request for a Meeting

Dear [Recipient Name],

I hope this email finds you well. I am writing to request a meeting with you at our company to discuss the following:

[Proposed topic]

This meeting will be a great opportunity for us to discuss the topic in detail, share our respective perspectives, an
d come up with a solution that meets both parties' needs.

I am confident that our meeting will be productive and fruitful, and I am looking forward to discussing this matter w
ith you.

Please let me know if you are available to meet with me on [date] at [time]. If you cannot make it, please let me kno
w

--- FINE-TUNED MODEL ---
Dear [Employee’s Name],

I am reaching out to schedule a meeting with you. I am [Insert specific reason for meeting here] and I would like to
discuss [Insert specific topic here]. This meeting is essential for [Insert specific reason for meeting here] and I b
elieve it will be a fruitful discussion.

Please let me know if you have any availability on [Insert date and time of the meeting here].

Thank you for your time and consideration.

Sincerely,
[Your Name]

================================================================================================
PROMPT:
What are three benefits of regular exercise?

--- BASE MODEL ---
1. Increases metabolism: Regular exercise increases your metabolism, which means your body burns calories at a faster
rate. This can help you lose weight and maintain a healthy weight over time.

2. Improves cardiovascular health: Regular exercise can help improve your cardiovascular health by increasing blood f
low to your muscles and reducing the risk of heart disease.

3. Boosts mood: Regular exercise has been shown to have a positive impact on mood and overall well-being. It can help
reduce stress and improve overall happiness.

These benefits are all part of the "benefits of exercise" that you can

--- FINE-TUNED MODEL ---
1. Improved Cardiovascular Health
2. Reduced Risk of Chronic Diseases
3. Increased Mental Health

================================================================================================
PROMPT:
Explain the difference between a list and a tuple in Python.

--- BASE MODEL ---
In Python, a list is a collection of items of the same type, where each item is stored in a separate position in the
list. In contrast, a tuple is a collection of items of the same type, where each item is stored in a single position
in the tuple.
```

```
Here are some examples of how they differ:

1. List vs. Tuple:
- Lists are immutable (meaning they cannot be modified after being created)
- Tuples are mutable (meaning they can be modified after being created)

2. Length and size of a list:
- A list can have a fixed length, but it cannot grow beyond that length
- A tuple can grow as long as

--- FINE-TUNED MODEL ---
A tuple is an immutable sequence of items, whereas a list is an mutable sequence of items. A tuple can be assigned mu
ltiple times, but a list cannot.
```

Done

You have successfully fine-tuned TinyLlama with LoRA on the Dolly dataset. Review your base vs fine-tuned outputs and note the differences in instruction-following quality.